

Критические секции, взаимное исключение.

При параллельной работе нескольких потоков на одном участке памяти могут возникнуть неприятные эффекты, связанные с переключением задач. Рассмотрим модельный пример выполнения операции

```
x += 1;
```

которая присутствует во всех потоках и относится к одной и той же области памяти, занятой переменной x . Например, x отвечает за некоторый глобальный счетчик.

Процессор реализует эту операцию некоторой последовательностью машинных команд и, если переключение между потоками вдруг возникнет между этими командами, то в результате итоговое значение x может быть вычисленно неверно. Разные архитектуры процессоров используют разные системы команд, но обычно вычисления происходят по общей схеме — загрузка данных в регистры процессора, выполнение операции на регистрах, выгрузка данных из регистров в память. Поэтому в некотором приближении данную операцию инкремента переменной можно описать, например, последовательностью трех условных команд

```
load a x    // загрузка x из памяти в регистр a
inc  a      // увеличение регистра на 1
stor a x    // выгрузка значения из регистра a в память по адресу x
```

(которые не являются реальными, но отражают идею работы).

Напомним, что процессор вместе со своими регистрами включается в контекст потока, т.е. при переключении потоков регистры сохраняются и потом восстанавливаются. Теперь рассмотрим варианты с переключением, имея в виду, что текущий поток у нас может быть прерван в любом месте (т.е. между любыми двумя командами)

```
поток A      поток B
load a x
inc  a
stor a x
.....
                load a x
                inc  a
                stor a x
                .....
.....
```

Здесь все в порядке. Поток А увеличил переменную x , прервался, и далее поток В в свою очередь увеличил эту переменную.

Теперь рассмотрим вот такой вариант:

```
поток A      поток B
load a x
                load a x
                inc  a
                stor a x
                .....
inc  a
stor a x
.....
```

здесь поток В вклинился в код для увеличения переменной. Оба потока получили в свои регистры одно и то же значение переменной x из памяти, увеличили его на 1 и сохранили. Более того, поток А переписал своим значением тот результат, который сохранил поток В.

Этот эффект связан с понятием критической секции, т.е. фрагмента кода, который может дать неверный результат, если его разорвать выполнением другого потока.

Как поток может обеспечить “неприкосновенность” данных, с которыми ему нужно сейчас работать? Можно было бы ввести специальную переменную m — флаг доступа. Установили $m = 1$ и работаем с данными, а другие потоки проверяют, если $m == 1$, то трогать эти данные нельзя, пока переменная не станет опять $m == 0$.

```

m = 1;
работаем с данными, никто их не будет трогать
m = 0;
теперь мы на эти данные не претендуем, их можно менять в других потоках

```

Однако простая проверка $if(m > 0)$ в другом потоке нас не спасет, поскольку такая проверка тоже раскладывается в серию команд типа

```

load a m          // загрузить m в регистр
tst  a            // выяснить какой знак у числа в регистре
jmp_pos addr      // перейти по адресу для кода с +
....             // код, если знак не положительный

```

И теперь если переключение произойдет сразу после проверки `tst`, то может быть проблема

<pre> процесс А load a m tst a (m==0 !) jmp_pos addr выполняем код для m==0 ... прервались </pre>	<pre> процесс В m = 0; m = 1; работаем с данными ... прервались продолжаем работать с данными ???</pre>
---	---

Таким образом, проверку подобного условия и переход к выполнению соответствующего ему кода нельзя прерывать переключением контекста.

Для реализации этого принципа есть несколько инструментов: `mutex`, семафоры, `atomic`. Пока рассмотрим простейший из них — `mutex` — двоичный флажок занято/свободно.

```

pthread_mutex_t
int pthread_mutex_init(pthread_mutex_t*, NULL); // второй арг. - атрибуты
int pthread_mutex_lock(pthread_mutex_t*);
int pthread_mutex_unlock(pthread_mutex_t*);
int pthread_mutex_trylock(pthread_mutex_t*);
int pthread_mutex_destroy(pthread_mutex_t*);

```

Все функции возвращают 0 при успехе и код ошибки при отказе.

Схема использования

```

pthread_mutex_t  m;
int pthread_mutex_init(&m, NULL);
// либо
pthread_mutex_t  m = PTHREAD_MUTEX_INITIALIZER;
...
pthread_mutex_lock(&m);
    критическая секция
pthread_mutex_unlock(&m);
...
pthread_mutex_destroy(&m);
.....
pthread_mutex_trylock(&m) - блокирует mutex если он unlock,
                          возвращает код ошибки, если lock и не меняет его
                          т.е. можно проверять на lock|unlock

```

Пример. Файл `parint-mutex.cpp` — многопоточное интегрирование, где каждый поток самостоятельно увеличивает итоговое значение интеграла.

Пример. Файл `rw0.cpp` — синхронизация двух потоков в задаче читателей-писателей (производителей-потребителей). Писатель постоянно создает новые тексты и выкладывает их во всеобщий доступ для ознакомления. Читатель следит за появлением новых текстов от писателя и, как только выходит новый текст, сразу его считывает и некоторое время над ним размышляет. Писатель выкладывает свои уникальные тексты “в одном экземпляре”, и только по одному. То есть, читатель “забирает текст к себе” и этот текст становится больше никому не доступным, а писатель получает возможность выложить в общий доступ новый текст. Таким образом, мы получаем дисциплину синхронизации: читатель может получить новый текст только после того как писатель выложил его в открытый доступ, а писатель может выложить новый текст только после того как читатель забрал предыдущий.

Здесь используется два `mutex`, один (`mut_read`) для организации монопольного доступа в буферу обмена текстом (а то вдруг писатель захочет записать новый текст в то время, когда читатель читает старый) и другой (`mut_write`), чтобы блокировать писателя до того момента, пока читатель не заберет предыдущий текст.

Задачи.

1. Найти максимальное значение в массиве. Отдельные потоки по частям массива в конце своей работы обновляют глобальный максимум. Общая переменная для искомого максимума. Критическая секция — обновление максимума. Проверка ускорения от числа потоков и зависимости от размера массива.

2. Задачу про писателей-читателей можно переформулировать как задачу производителей-потребителей.

Производитель изготавливает некоторый товар и отправляет его на склад (в магазин, и т.п.). Склад имеет ограниченный размер, в нем помещается не более заданного количества единиц товара. Если на складе мест уже нет, то производитель вынужден простаивать, пока места не освободятся. Потребитель берет со склада товар, если он там есть. Если нет, то ожидает, когда товар появится.

Требуется смоделировать взаимодействие производителя и потребителя в разных моделях поведения, которые могут включать следующее:

- производитель и потребитель могут пытаться выкладывать/забирать товар через разные (фиксированные или случайные) промежутки времени;
- производителей и потребителей может быть несколько (в разных комбинациях: $1 : m$, $n : 1$, $n : m$);
- потребитель может запросить не одну, а несколько единиц товара;
- товар имеет срок годности, если его не забрали за указанный период времени, то он удаляется со склада.

Результатом моделирования должно быть некоторое описание (протокол) того, как этот процесс разворачивался во времени, соблюдалась ли требуемая дисциплина и т.п. Также интересны суммарные характеристики, которые могут быть значимы для реального производства — сколько времени простаивал производитель, сколько времени ожидал потребитель, как долго товары находились на складе невостребованными и т.п.

3. Классическая задача обслуживания потока запросов. Источник генерирует последовательность запросов (для простоты пусть запрос — это целое число). Приемник принимает запросы и складывает их в очередь. Обработчик берет запросы из очереди и обрабатывает их (тратит на каждый запрос некоторое время). Источник, приемник и обработчик — это разные потоки. Проблема в том, что поток запросов может быть неравномерным во времени, иногда запросы следуют часто друг за другом, а иногда их долго нет. Если обрабатывать запросы сразу по мере их поступления, то при интенсивном потоке обработчик может не успеть обработать текущий запрос до поступления следующего. Очередь здесь является накопителем пришедших, но пока еще не обслуженных запросов. А если поток запросов ослаб, то, наоборот, обработчик будет простаивать. Можно моделировать различные вероятностные распределения интервалов между приходом запросов и смотреть успевает ли ваша система обрабатывать все запросы, какой размер очереди для этого требуется и т.п.

При решении имеет смысл воспользоваться классом очереди из библиотеки STL.